4. Differentiation and Quandrature

4.1. Introduction

4.1.1. Motivation

In many scientific and engineering problems, we often encounter functions that are either too complex or not available in analytic form at all. For example, physical quantities may be described by empirical data points (from experiments or simulations), tabulated values, or functions defined only implicitly. Calculating derivatives or integrals of such functions exactly is impossible without an explicit formula. Even when an analytic expression exists, the derivative or integral might be too complicated to compute by hand or even symbolically. Therefore, practical numerical methods are essential for approximating both derivatives and integrals, allowing us to extract meaningful information from real-world data or complex models.

4.1.2. Examples

Work Done by a Variable Force

In classical mechanics, the work done by a force along a path is given by the integral of the force over distance. When the force varies with position in a complex or non-analytic way, such as when a spacecraft moves through a non-uniform gravitational field, the integral cannot be solved analytically and must be approximated numerically.

$$W = \int_{x_0}^{x_1} F(x)\,dx \tag{4.1}$$

Here, $F(x)$ is the position-dependent force. If $F(x)$ is known only at discrete points, interpolation and numerical integration become necessary, see left-hand panel in Fig. 4.1.

Total Charge on an Object with Non-Uniform Charge Density

In electrostatics, the total electric charge on an object depends on the integral of the charge density over its volume (or length/area, depending on the object). For a rod or surface with a non-uniform charge density that varies in a complicated way or is known only at measured data points, numerical methods are required to approximate the total charge.

$$Q = \int_{a}^{b} \rho(x)\,dx \tag{4.2}$$

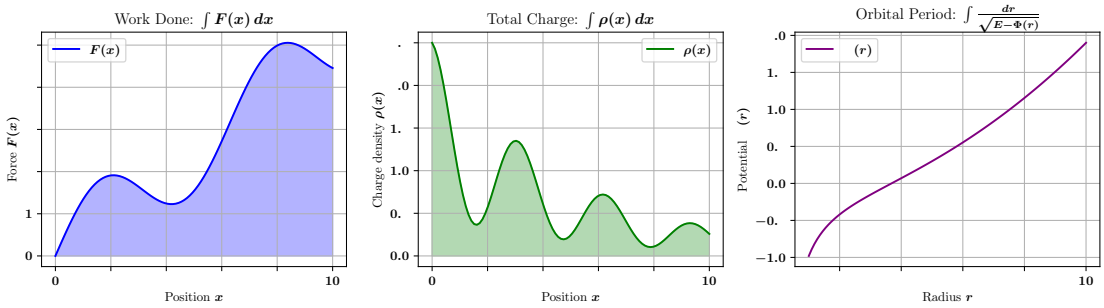

Figure 4.1.: Illustrative examples of physical integrals. **Left:** The work done by a variable force $F(x)$ is given by the area under the force curve between x_0 and x_1. The shaded region visually represents this integral. **Center:** The total electric charge Q on an object with non-uniform charge density $\rho(x)$ corresponds to the area under the $\rho(x)$ curve. Again, the shaded region highlights the integral. **Right:** The orbital period T depends on an integral involving the inverse square root of the energy difference and the gravitational potential $\Phi(r)$. The plot shows the potential $\Phi(r)$, there is no shaded area because the period integral depends on a different function derived from $\Phi(r)$, not on the potential itself.

where $\rho(x)$ is the charge density along the object. If $\rho(x)$ is given by measurements or simulations at discrete points, interpolation and numerical integration must be applied, see middle panel in Fig. 4.1.

Orbital Period in a Non-Keplerian Gravitational Potential

In orbital mechanics, the period of an object orbiting in a central potential depends on the shape of that potential. For simple cases like a point mass, the period can be calculated analytically using Kepler's laws. However, in more realistic scenarios (such as galaxies or planets with extended mass distributions), the gravitational potential $\Phi(r)$ can vary in complex ways. Computing the orbital period then requires numerical integration.

$$T = 2\int_{r_{\min}}^{r_{\max}} \frac{dr}{\sqrt{2(E-\Phi(r)) - \frac{L^2}{r^2}}} \tag{4.3}$$

where E is the total energy and L is the angular momentum. If $\Phi(r)$ is known only numerically or at discrete points, interpolation and numerical integration are necessary to compute T, see right-hand panel in Fig. 4.1.

4.1.3. Unified approach

A powerful and unifying strategy for tackling both numerical differentiation and numerical integration is to first approximate the unknown function using an interpolation

polynomial. Once we have a polynomial approximation, taking derivatives or integrals becomes straightforward because polynomials are easy to differentiate and integrate exactly. Among various interpolation methods, Lagrange interpolation polynomials are particularly useful because they provide a clear, constructive way to interpolate a function using a set of known data points. By differentiating the interpolating polynomial, we obtain numerical differentiation formulas such as the three-point and five-point midpoint rules. Similarly, by integrating the interpolant, we can derive numerical quadrature rules, including the trapezoidal rule and Simpson's rule. This approach not only provides practical algorithms but also offers deep insight into how numerical differentiation and integration methods are connected through the fundamental concept of interpolation.

4.2. Numerical Differentiation

The definition of the derivative of the function f at x_0 is

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h}. \tag{4.4}$$

Based on this formula is seems obvious to compute an approximation to $f'(x_0)$ via

$$\frac{f(x_0 + h) - f(x_0)}{h} \tag{4.5}$$

for small values of h. This is of course a good point to start from, but can quickly lead to inccorect values due to round-off and cancellation errors. Also, we might not have function values *as close as possible*, so if the values are only known at certain points, we cannot simply chose a small h as we like.

4.2.1. Review of Lagrange Interpolation Polynomials

Given $n + 1$ data points $(x_0, f_0), (x_1, f_1), \ldots, (x_n, f_n)$, the **Lagrange interpolation polynomial** of degree n is given by:

$$P_n(x) = \sum_{i=0}^{n} f_i \cdot L_i(x), \quad \text{where} \quad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j} \tag{4.6}$$

are the **Lagrange basis polynomials**. The first three lowest order degree Lagrange polynomials are a constant function passing through the single point (x_0, f_0) with degree $n = 0$,

$$P_0(x) = f_0, \tag{4.7}$$

a straight line passing through the two points (x_0, f_0) and (x_1, f_1) with $n = 1$,

$$P_1(x) = f_0 \cdot \frac{x - x_1}{x_0 - x_1} + f_1 \cdot \frac{x - x_0}{x_1 - x_0}, \tag{4.8}$$

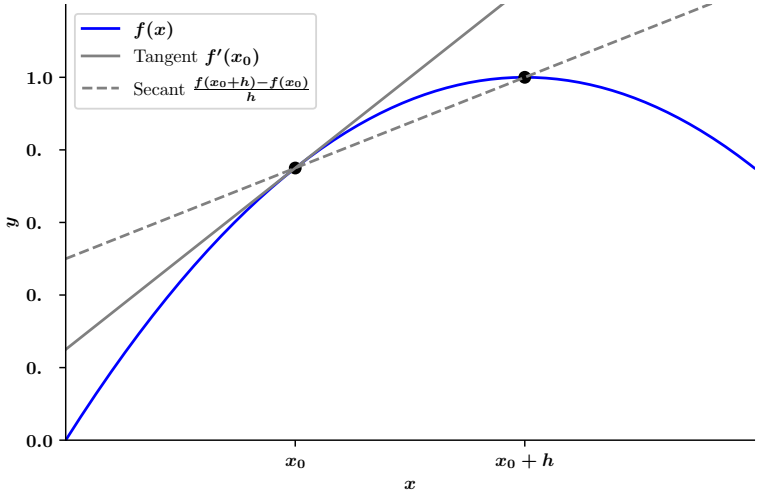

Figure 4.2.: Approximation of the first derivative $f'(x_0)$ with the *forward-difference* formula.

and a parabula for second-degree interpolation ($n = 2$):

$$P_2(x) = f_0 \cdot \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} + f_1 \cdot \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} + f_2 \cdot \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)}. \quad (4.9)$$

We also recap the error polynomial:

$$f(x) = P_n(x) + \frac{f^{(n+1)}(\xi)}{(n+1)!} \cdot \omega_{n+1}(x), \quad \text{where} \quad \omega_{n+1}(x) = \prod_{i=0}^{n} (x - x_i). \quad (4.10)$$

is the **error polynomial** with $\xi(x) \in [x_0, x_1]$.

4.2.2. Construction of Lagrange polynomials

To ensure we are on mathematically well founded ground for an approximation of $f'(x_0)$, suppose that $x_0 \in (a, b)$, $f \in C^2[a, b]$, and $x_1 = x_0 + h$ for a small $h \neq 0$, such that $x_1 \in [a, b]$. We can then construct the first-order Lagrange polynomial $P_1(x)$ for f determined by x_0 and x_1, with its error term:

$$f(x) = P_1(x) + \frac{(x - x_0)(x - x_1)}{2!} f''(\xi(x)) \quad (4.11)$$

$$= \frac{f(x_0)(x - x_1)}{-h} + \frac{f(x_0 + h)(x - x_0)}{h} + \frac{(x - x_0)(x - x_0 - h)}{2} f''(\xi(x)). \quad (4.12)$$

Differentiation with respect to x, D_x, yields

$$f'(x) = \frac{f(x_0+h)-f(x_0)}{h} + D_x\left[\frac{(x-x_0)(x-x_0-h)}{2}f''(\xi(x))\right] \tag{4.13}$$

$$= \frac{f(x_0+h)-f(x_0)}{h} + \frac{2(x-x_0)-h}{2}f''(\xi(x)) \tag{4.14}$$

$$+ \frac{(x-x_0)(x-x_0-h)}{2}D_x[f''(\xi(x))].$$

If we forget about the error term – remove terms involving $f''(\xi(x))$, $D_x\left[f''(\xi(x))\right]$ – we find:

$$f'(x) \approx \frac{f(x_0+h)-f(x_0)}{h}, \tag{4.15}$$

which is the same as the intuitively computed approach. One problem with this formula is that we do not know $D_x\left[f''(\xi(x))\right]$, so the truncation error cannot be estimated for general x. However, at the interpolation point $x = x_0$, the error term involving $D_x\left[f''(\xi(x))\right]$ vanishes by construction because the interpolation polynomial is required to pass exactly through $f(x_0)$, making the error of the derivative zero at that point. As a result, the formula simplifies to:

$$f'(x_0) = \frac{f(x_0+h)-f(x_0)}{h} - \frac{h}{2}f''(\xi). \tag{4.16}$$

This means, we can approximate the derivative via $\left[f(x_0+h)-f(x_0)\right]/h$ for small h with an error bounded by $|f''(x)|\,|h|/2$, between x_0 and x_0+h. This formula is known as the **forward-difference formula** if $h > 0$ (see Fig. 4.2) and the **backward-difference formula** if $h < 0$.

4.2.3. Higher order polynomials

We can extend this to a general approximation for $n+1$ points $\{x_0, x_1, \ldots, x_n\}$. The equation for the approximating interpolation polynomial reads

$$f(x) = \underbrace{\sum_{k=0}^{n} f(x_k)L_k(x)}_{\text{interpol. polynom.}} + \underbrace{\frac{(x-x_0)\cdots(x-x_n)}{(n+1)!}f^{(n+1)}(\xi(x))}_{\text{error polynomial}}, \tag{4.17}$$

again for some $\xi(x)$ and the kth Lagrange coefficient polynomial $L_k(x)$ at $x_0, x_1, \ldots, x_n$. By differentiating this expression we arrive at

$$f'(x) = \sum_{k=0}^{n} f(x_k)L_k'(x) + D_x\left[\frac{(x-x_0)\cdots(x-x_n)}{(n+1)!}\right]f^{(n+1)}(\xi(x)) \tag{4.18}$$

$$+ \frac{(x-x_0)\cdots(x-x_n)}{(n+1)!}D_x\left[f^{(n+1)}(\xi(x))\right].$$

In this general form we have again the problem of the unknown truncation error, except at the locations of the nodes x_j. Here, the term with $D_x[f^{(n+1)}(\xi(x))]$ is 0, so we only need to worry about the term

$$D_x \left[\frac{(x - x_0) \cdots (x - x_n)}{(n+1)!} \right]. \tag{4.19}$$

Let

$$Q(x) = (x - x_0)(x - x_1) \cdots (x - x_n) \tag{4.20}$$

Then

$$D_x \left[\frac{Q(x)}{(n+1)!} \right] = \frac{1}{(n+1)!} \cdot Q'(x) \tag{4.21}$$

and for the derivative of $Q(x)$ we apply the product rule, so for each k, differentiate $(x - x_k)$ (which gives 1), and multiply by all other factors,

$$Q'(x) = \sum_{i=0}^{n} \left[\prod_{\substack{k=0 \\ k \neq i}}^{n} (x - x_i) \right] \tag{4.22}$$

Now we evaluate Q' at $x = x_j$,

$$Q'(x_j) = \sum_{i=0}^{n} \left[\prod_{\substack{k=0 \\ k \neq i}}^{n} (x_j - x_i) \right] \tag{4.23}$$

At $x = x_j$, every term in the sum except for $i = j$ vanishes, since if $i \neq j$, the product contains $(x_i - x_i) = 0$. Thus

$$Q'(x_j) = \prod_{\substack{k=0 \\ k \neq j}}^{n} (x_j - x_k) \tag{4.24}$$

and

$$D_x \left[\frac{(x - x_0)(x - x_1) \cdots (x - x_n)}{(n+1)!} \right]_{x = x_j} = \frac{1}{(n+1)!} \cdot \prod_{\substack{k=0 \\ k \neq j}}^{n} (x_j - x_k) \tag{4.25}$$

With this result we arrive at the so-called **(n+1)-point formula** to approximate $f'(x_j)$.

$$f'(x_j) = \sum_{k=0}^{n} f(x_k) L'_k(x_j) + \frac{f^{(n+1)}(\xi(x_j))}{(n+1)!} \prod_{\substack{k=0 \\ k \neq j}}^{n} (x_j - x_k). \tag{4.26}$$

In general, using more evaluation points in Eq. (4.2) can produce greater accuracy because higher-degree interpolating polynomials capture the behavior of the function more precisely. However, increasing the number of points also raises the number of function evaluations and can lead to greater sensitivity to round-off errors and numerical instability. As with other numerical methods, this reflects the common tradeoff between higher-order formulas for improved accuracy and lower-order formulas for greater stability and robustness. **For practical purposes, the most widely used formulas typically involve three or five evaluation points.**

We begin by deriving some useful three-point formulas and examining their associated errors. For

$$L_0(x) = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)}, \tag{4.27}$$

we find

$$L_0'(x) = \frac{2x - x_1 - x_2}{(x_0 - x_1)(x_0 - x_2)}. \tag{4.28}$$

Analogously, we get

$$L_1'(x) = \frac{2x - x_0 - x_2}{(x_1 - x_0)(x_1 - x_2)} \quad \text{and} \quad L_2'(x) = \frac{2x - x_0 - x_1}{(x_2 - x_0)(x_2 - x_1)}, \tag{4.29}$$

which we can insert in Eq. (4.26) to get

$$\begin{aligned} f'(x_j) = f(x_0)\left[\frac{2x_j - x_1 - x_2}{(x_0 - x_1)(x_0 - x_2)}\right] + f(x_1)\left[\frac{2x_j - x_0 - x_2}{(x_1 - x_0)(x_1 - x_2)}\right] \\ + f(x_2)\left[\frac{2x_j - x_0 - x_1}{(x_2 - x_0)(x_2 - x_1)}\right] + \frac{1}{6}f^{(3)}(\xi_j)\prod_{\substack{k=0 \\ k\neq j}}^{2}(x_j - x_k), \end{aligned} \tag{4.30}$$

for each $j = 0, 1, 2$, and ξ_j indicating that it depends on x_j.

4.2.4. Three-Point Formulas

An efficient simplification is to assume equally spaced nodes

$$x_1 = x_0 + h \quad \text{and} \quad x_2 = x_0 + 2h, \quad \text{for } h \neq 0, \tag{4.31}$$

which we will assume for the remainder of this chapter. We can investigate Eq.(4.30) for the forward, centred and backward version:

- for $x_j = x_0$, $x_1 = x_0 + h$, and $x_2 = x_0 + 2h$ (*forward*):

$$f'(x_0) = \frac{1}{h}\left[-\frac{3}{2}f(x_0) + 2f(x_1) - \frac{1}{2}f(x_2)\right] + \frac{h^2}{3}f^{(3)}(\xi_0) \tag{4.32}$$

$$= \frac{1}{h}\left[-\frac{3}{2}f(x_0) + 2f(x_0 + h) - \frac{1}{2}f(x_0 + 2h)\right] + \frac{h^2}{3}f^{(3)}(\xi_0) \tag{4.33}$$

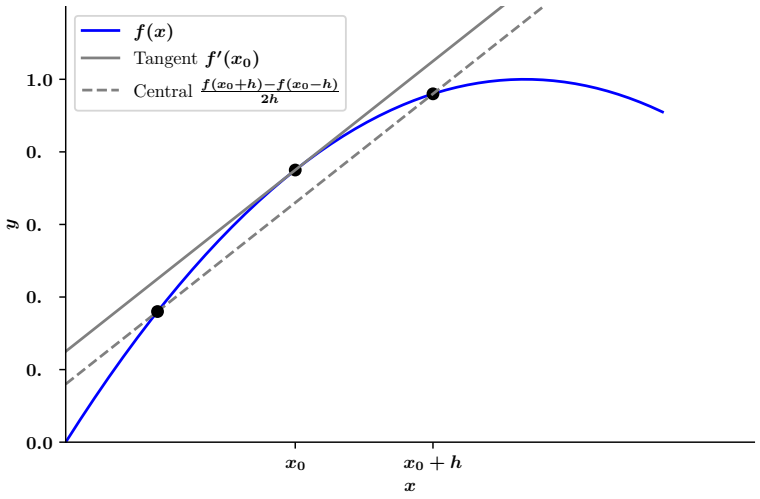

Figure 4.3.: Approximation of the first derivative $f'(x_0)$ with the *3-point midpoint* formula.

- The same for $x_j = x_1$ results in the *centred* version,

$$f'(x_1) = \frac{1}{h}\left[-\frac{1}{2}f(x_0) + \frac{1}{2}f(x_2)\right] \tag{4.34}$$

$$= \frac{1}{h}\left[-\frac{1}{2}f(x_0) + \frac{1}{2}f(x_0+2h)\right] - \frac{h^2}{6}f^{(3)}(\xi_1), \tag{4.35}$$

  which is illustrated in Fig. 4.3

- and $x_j = x_2$ gives the *backward* solution,

$$f'(x_2) = \frac{1}{h}\left[\frac{1}{2}f(x_0) - 2f(x_1) + \frac{3}{2}f(x_2)\right] + \frac{h^2}{3}f^{(3)}(\xi_2) \tag{4.36}$$

$$= \frac{1}{h}\left[\frac{1}{2}f(x_0) - 2f(x_0+h) + \frac{3}{2}f(x_0+2h)\right] + \frac{h^2}{3}f^{(3)}(\xi_2). \tag{4.37}$$

It is convenient to name the node that is the centre of the evaluation x_0, so the commonly used derivatives are

Three-Point Endpoint Formula

$$f'(x_0) = \frac{1}{2h}\left[-3f(x_0) + 4f(x_0+h) - f(x_0+2h)\right] + \frac{h^2}{3}f^{(3)}(\xi_0) \tag{4.38}$$

with ξ_0 between x_0 and $x_0 + 2h$.

Three-Point Midpoint Formula

$$f'(x_0) = \frac{1}{2h}\left[f(x_0+h) - f(x_0-h)\right] - \frac{h^2}{6}f^{(3)}(\xi_1), \tag{4.5}$$

with ξ_1 between $x_0 - h$ and $x_0 + h$.

Three-Point Startpoint Formula

$$f'(x_0) = \frac{1}{2h}\left[f(x_0-2h) - 4f(x_0-h) + 3f(x_0)\right] + \frac{h^2}{3}f^{(3)}(\xi_2) \tag{4.39}$$

with ξ_0 in $x_0 - 2h$ and x_0

4.2.5. Higher order polynomials and higher order derivatives

Five-Point Midpoint Formula

We can extend this method to 5th order order and get

$$f'(x_0) = \frac{1}{12h}\left[f(x_0-2h) - 8f(x_0-h) + 8f(x_0+h) - f(x_0+2h)\right] + \frac{h^4}{30}f^{(5)}(\xi), \tag{4.40}$$

with ξ between $x_0 - 2h$ and $x_0 + 2h$.

Second Derivative Midpoint Formula

Instead of going to higher order in the number of nodes and the resulting degree of the polynomial, we can also increase the order of the derivative. For the second derivative a commonly used formulation is

$$f''(x_0) = \frac{1}{h^2}\left[f(x_0-h) - 2f(x_0) + f(x_0+h)\right] - \frac{h^2}{12}f^{(4)}(\xi), \tag{4.41}$$

for $x_0 - h < \xi < x_0 + h$. If $f^{(4)}$ is continuous on $[x_0 - h, x_0 + h]$, it is also bounded, and the approximation is $O(h^2)$.

4.3. Numerical Integration / Quadrature

4.3.1. Basic idea

The basic method involved in approximating $\int_a^b f(x)\mathrm{d}x$ is called **numerical quadrature**, which uses a sum $\sum_{i=0}^n a_i f(x_i)$ to approximate the integral.

For this task we come back again to interpolation polynomials at a set of distinct nodes $x_0, \ldots, x_n$ in the interval $[a, b]$. We start again with the Lagrange interpolation polynomial

$$P_n(x) = \sum_{i=0}^{n} f(x_i)L_i(x) \tag{4.42}$$

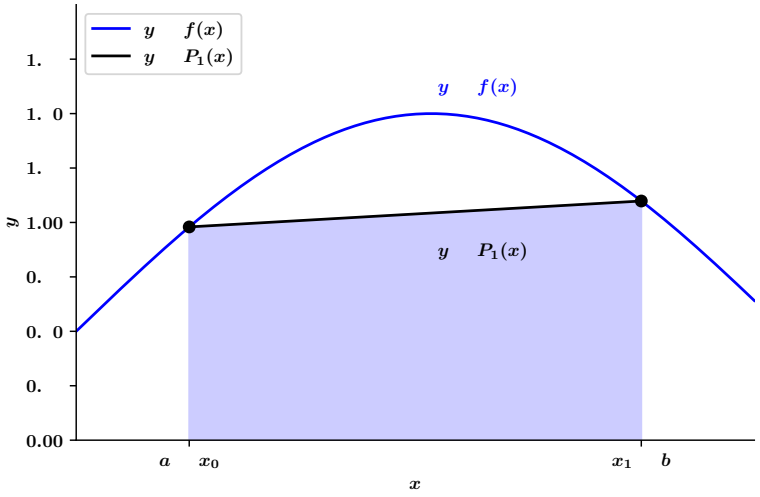

Figure 4.4.: Illustration of the Trapezoidal Rule.

and its truncation error term over $[a, b]$ to obtain

$$\int_a^b f(x)\,dx = \int_a^b \sum_{i=0}^{n} f(x_i)L_i(x)\,dx + \int_a^b \prod_{i=0}^{n}(x - x_i)\,\frac{f^{(n+1)}(\xi(x))}{(n+1)!}\,dx \tag{4.43}$$

$$= \sum_{i=0}^{n} a_i f(x_i) + \frac{1}{(n+1)!}\int_a^b \prod_{i=0}^{n}(x - x_i)\,f^{(n+1)}(\xi(x))\,dx \tag{4.44}$$

where $\xi(x)$ is in $[a, b]$ for each x and

$$a_i = \int_a^b L_i(x)\,dx, \quad \text{for each } i = 0, 1, \ldots, n. \tag{4.45}$$

The quadrature formula is, therefore,

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n} a_i f(x_i), \tag{4.46}$$

with error given by

$$E(f) = \frac{1}{(n+1)!}\int_a^b \prod_{i=0}^{n}(x - x_i)\,f^{(n+1)}(\xi(x))\,dx. \tag{4.47}$$

4.3.2. Trapezoidal Rule

We start with the first order Lagrange polynomial and define $x_0 = a$, $x_1 = b$, $h = b - a$,

$$P_1(x) = \frac{(x - x_1)}{(x_0 - x_1)}f(x_0) + \frac{(x - x_0)}{(x_1 - x_0)}f(x_1). \tag{4.48}$$

Then

$$\int_a^b f(x)\,dx = \int_{x_0}^{x_1} \left[\frac{(x-x_1)}{(x_0-x_1)}f(x_0) + \frac{(x-x_0)}{(x_1-x_0)}f(x_1)\right] dx + \frac{1}{2}\int_{x_0}^{x_1} f''(\xi(x))(x-x_0)(x-x_1)\,dx. \tag{4.23}$$

The product $(x-x_0)(x-x_1)$ does not change sign on $[x_0, x_1]$, so the Weighted Mean Value Theorem for Integrals can be applied to the error term to give, for some $\xi \in (x_0, x_1)$,

$$\int_{x_0}^{x_1} f''(\xi(x))(x-x_0)(x-x_1)\,dx = f''(\xi)\int_{x_0}^{x_1}(x-x_0)(x-x_1)\,dx \tag{4.49}$$

$$= f''(\xi)\left[\frac{x^3}{3} - \frac{(x_1+x_0)}{2}x^2 + x_0x_1x\right]_{x_0}^{x_1} \tag{4.50}$$

$$= -\frac{h^3}{6}f''(\xi). \tag{4.51}$$

Consequently, Eq. (4.23) implies that

$$\int_a^b f(x)\,dx = \left[\frac{(x-x_1)^2}{2(x_0-x_1)}f(x_0) + \frac{(x-x_0)^2}{2(x_1-x_0)}f(x_1)\right]_{x_0}^{x_1} - \frac{h^3}{12}f''(\xi)$$

$$= \frac{(x_1-x_0)}{2}\left[f(x_0)+f(x_1)\right] - \frac{h^3}{12}f''(\xi).$$

Using $h = x_1 - x_0$ yields the **Trapezoidal Rule**

$$\int_a^b f(x)\mathrm{d}x = \frac{h}{2}\left[f(x_0)+f(x_1)\right] - \frac{h^3}{12}f''(\xi), \tag{4.52}$$

because for positive function values f the integral $\int_a^b f(x)\,dx$ is approximated by the area in a trapezoid, as illustrated in Fig. 4.4.

The error term scales with f'', so for all function whose second derivative is identically zero – so for any polynomial of degree one or less – the rule gives the exact result.

4.3.3. Simpson's Rule

Simpson's rule starts by integrating over $[a, b]$ using the second Lagrange polynomial defined on equally spaced nodes $x_0 = a$, $x_2 = b$, and $x_1 = a + h$, with $h = (b-a)/2$, see Fig. 4.5. We derive the following terms for the integral,

$$\int_a^b f(x)\,dx = \int_{x_0}^{x_2} \left[\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}f(x_0) + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}f(x_1) + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}f(x_2)\right] dx + \int_{x_0}^{x_2}\frac{(x-x_0)(x-x_1)(x-x_2)}{6}f^{(3)}(\xi(x))\,dx. \tag{4.53}$$

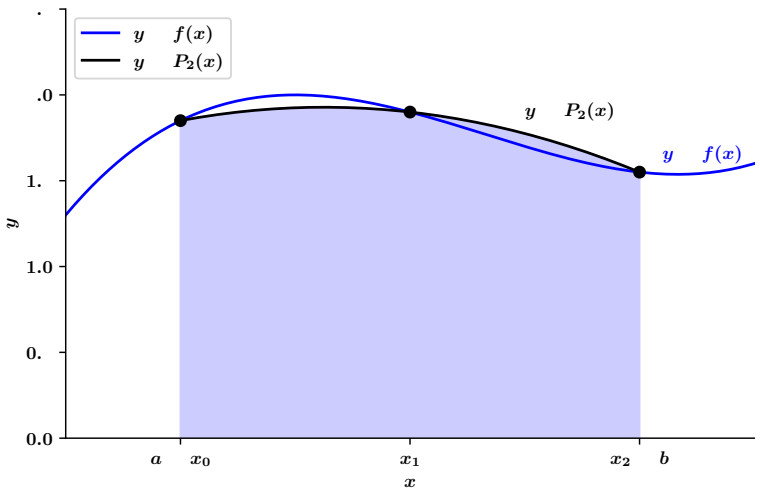

Figure 4.5.: Illustration of Simpson's Rule

Computing the individual components like this is, however, tedious and provides only an $O(h^4)$ error term involving the third derivative, $f^{(3)}$. We therefore approach the problem in a different way using a higher-order term involving $f^{(4)}$.

For this, suppose that f is expanded in the third Taylor polynomial about x_1. Then, for each x in $[x_0, x_2]$, a number $\xi(x)$ in (x_0, x_2) exists with

$$f(x) = f(x_1)+f'(x_1)(x-x_1)+\frac{f''(x_1)}{2}(x-x_1)^2+\frac{f^{(3)}(x_1)}{6}(x-x_1)^3+\frac{f^{(4)}(\xi(x))}{24}(x-x_1)^4. \tag{4.54}$$

We can integrate this and derive

$$\begin{aligned}\int_{x_0}^{x_2} f(x)\,dx = &\left[f(x_1)x + \frac{f'(x_1)}{2}(x-x_1)^2 + \frac{f''(x_1)}{6}(x-x_1)^3 + \frac{f^{(3)}(x_1)}{24}(x-x_1)^4\right]_{x_0}^{x_2} \\ &+ \frac{1}{24}\int_{x_0}^{x_2} f^{(4)}(\xi(x))(x-x_1)^4\,dx.\end{aligned} \tag{4.55}$$

Since $(x-x_1)^4$ is never negative on $[x_0, x_2]$, we can find – based on the Weighted Mean Value Theorem for Integrals – a number ξ_1 in (x_0, x_2), so that the error terms is

$$\frac{1}{24}\int_{x_0}^{x_2} f^{(4)}(\xi(x))(x-x_1)^4\,dx = \frac{f^{(4)}(\xi_1)}{24}\int_{x_0}^{x_2}(x-x_1)^4\,dx = \frac{f^{(4)}(\xi_1)}{120}(x_2-x_1)^5. \tag{4.56}$$

The uniformly defined spacing between the points $h = x_2 - x_1 = x_1 - x_0$ gives

$$(x_2-x_1)^2-(x_0-x_1)^2 = (x_2-x_1)^4-(x_0-x_1)^4 = 0, \tag{4.57}$$

and

$$(x_2 - x_1)^3 - (x_0 - x_1)^3 = 2h^3 \quad \text{and} \quad (x_2 - x_1)^5 - (x_0 - x_1)^5 = 2h^5. \tag{4.58}$$

Consequently, Eq. (4.55) can be simplified to

$$\int_{x_0}^{x_2} f(x)\,dx = 2hf(x_1) + \frac{h^3}{3} f''(x_1) + \frac{f^{(4)}(\xi_1)}{60} h^5. \tag{4.59}$$

This equation now includes the second derivative, which we have approximated in the previous section, Eq. (4.41). Inserting $f''(x_1)$ yields

$$\begin{aligned}\int_{x_0}^{x_2} f(x)\,dx &= 2hf(x_1) + \frac{h^3}{3}\left[\frac{1}{h^2}\left(f(x_0) - 2f(x_1) + f(x_2)\right)\right] - \frac{h^5}{12} f^{(4)}(\xi_2) + \frac{f^{(4)}(\xi_1)}{60} h^5 \\ &= h\left[\frac{1}{3}f(x_0) + \frac{4}{3}f(x_1) + \frac{1}{3}f(x_2)\right] - \frac{h^5}{12}\left[\frac{1}{3}f^{(4)}(\xi_2) - \frac{1}{5}f^{(4)}(\xi_1)\right].\end{aligned}$$

This gives Simpson's rule.

$$\int_{x_0}^{x_2} f(x)\,dx = \frac{h}{3}\left[f(x_0) + 4f(x_1) + f(x_2)\right] - \frac{h^5}{90} f^{(4)}(\xi). \tag{4.60}$$

The error term in Simpson's rule involves the fourth derivative of f, so it gives exact results when applied to any polynomial of degree three or less.

4.4. Newton-Cotes formulas (NC)

We now introduce a generalization for higher degree polynomials, which are generally known as Newton-Cotes formulae. Here, we again assume equally spaced nodes on the interval $[a, b]$. We distinguished between two classes: *Closed* and *Open* Newton-Cotes formulae.

4.4.1. Closed NC

The $(n+1)$-point closed NC formulae use equally spaced points $x_i = x_0 + ih$, for $i = 0 \ldots n$ with $x_0 = a$, $x_n = b$, and $h = (b - a)/n$. They are called closed because the endpoint of the interval are included as nodes. This is illustrated and Fig. 4.6. The formulae are derived via

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n} a_i f(x_i), \tag{4.61}$$

where

$$a_i = \int_{x_0}^{x_n} L_i(x)\,dx = \int_{x_0}^{x_n} \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{(x - x_j)}{(x_i - x_j)}\,dx. \tag{4.62}$$

This leads to the following often used methods:

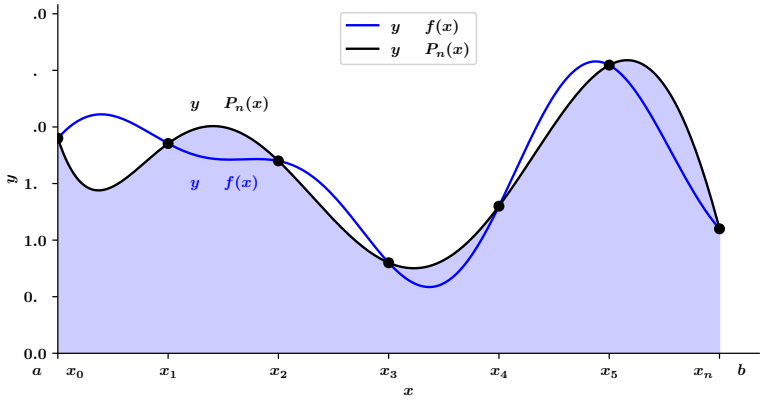

Figure 4.6.: Illustration of closed Newton-Cotes integration

$n = 1$: **Trapezoidal rule**

$$\int_{x_0}^{x_1} f(x)\,dx = \frac{h}{2}[f(x_0) + f(x_1)] - \frac{h^3}{12}f''(\xi), \quad \text{where} \quad x_0 < \xi < x_1. \tag{4.63}$$

$n = 2$: **Simpson's rule**

$$\int_{x_0}^{x_2} f(x)\,dx = \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)] - \frac{h^5}{90}f^{(4)}(\xi),$$
$$\text{where} \quad x_0 < \xi < x_2. \tag{4.64}$$

$n = 3$: **Simpson's Three-Eighths rule**

$$\int_{x_0}^{x_3} f(x)\,dx = \frac{3h}{8}[f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)] - \frac{3h^5}{80}f^{(4)}(\xi),$$
$$\text{where} \quad x_0 < \xi < x_3. \tag{4.65}$$

$n = 4$: **Boole's Rule**

$$\int_{x_0}^{x_4} f(x)\,dx = \frac{2h}{45}\left[7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)\right] - \frac{8h^7}{945}f^{(6)}(\xi),$$
$$\text{where} \quad x_0 < \xi < x_4. \tag{4.66}$$

4.4.2. Open NC

The *open Newton-Cotes formulas* skip the endpoints of $[a, b]$ and only use nodes inside the interval as illustrated in Fig. 4.7. The nodes are given by $x_i = x_0 + ih$, for $i = 0, 1, \ldots, n$, where $h = \frac{b-a}{n+2}$ and $x_0 = a+h$. This means the last node is at $x_n = b-h$. For convenience,

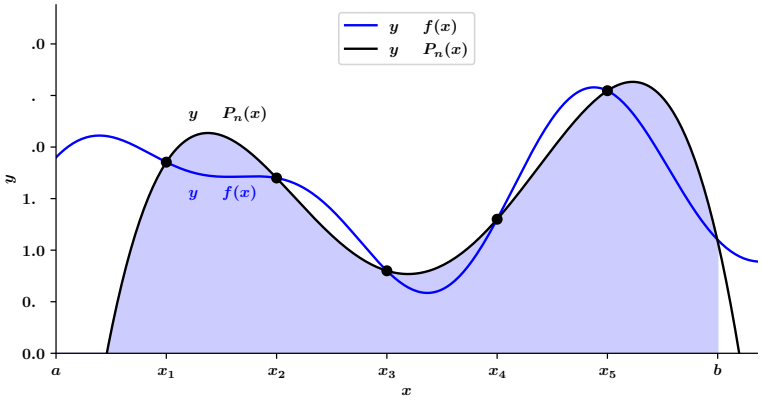

Figure 4.7.: Illustration of open Newton-Cotes integration

we also label the interval endpoints as $x_{-1} = a$ and $x_{n+1} = b$, as shown in Figure 4.6. All the nodes used in the approximation lie strictly between a and b. The formula can then be written as

$$\int_a^b f(x)\,dx = \int_{x_{-1}}^{x_{n+1}} f(x)\,dx \approx \sum_{i=0}^{n} a_i f(x_i), \tag{4.67}$$

where

$$a_i = \int_a^b L_i(x)\,dx. \tag{4.68}$$

Some often used versions of the open NC forumlae are:

$n = 0$: **Midpoint rule**

$$\int_{x_{-1}}^{x_1} f(x)\,dx = 2hf(x_0) + \frac{h^3}{3} f''(\xi), \quad \text{where} \quad x_{-1} < \xi < x_1. \tag{4.69}$$

$n = 1$:

$$\int_{x_{-1}}^{x_2} f(x)\,dx = \frac{3h}{2}[f(x_0) + f(x_1)] + \frac{3h^3}{4} f''(\xi), \quad \text{where} \quad x_{-1} < \xi < x_2. \tag{4.70}$$

$n = 2$:

$$\int_{x_{-1}}^{x_3} f(x)\,dx = \frac{4h}{3}\left[2f(x_0) - f(x_1) + 2f(x_2)\right] + \frac{14h^5}{45} f^{(4)}(\xi), \quad \text{where} \quad x_{-1} < \xi < x_3. \tag{4.71}$$

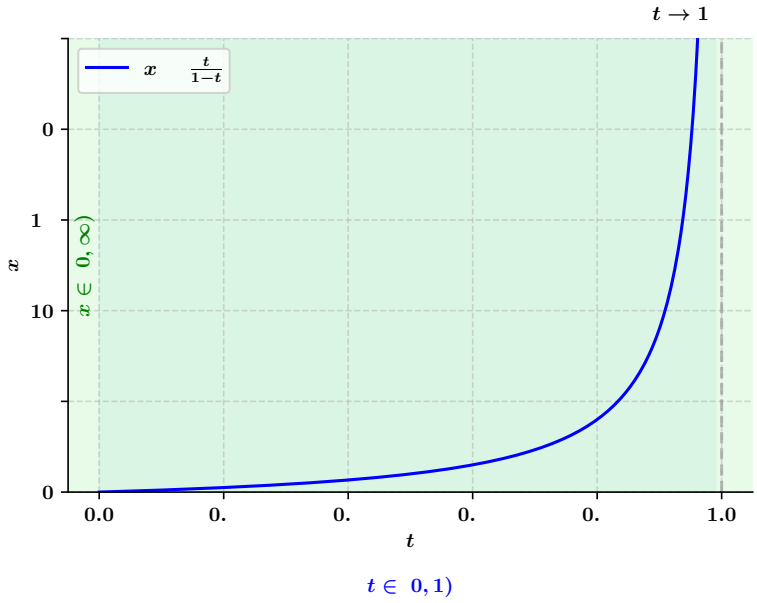

Figure 4.8.: Illustration of the transformation of a semi-infinite interval.

$n = 3$**:**

$$\int_{x_{-1}}^{x_4} f(x)\,dx = \frac{5h}{24}\left[11f(x_0) + f(x_1) + f(x_2) + 11f(x_3)\right] + \frac{95}{144}h^5 f^{(4)}(\xi), \quad \text{where} \quad x_{-1} < \xi < x_4. \tag{4.72}$$

4.4.3. Semi-infinite integrals

Suppose we want to compute integrals over a semi-infinite interval, for example:

$$\int_0^\infty f(x)\,dx. \tag{4.73}$$

Since Newton-Cotes formulae are only defined for finite intervals, we first transform the infinite interval to a finite one. A common change of variables is:

$$x = \frac{t}{1-t}, \quad \text{which maps } t \in [0,1) \text{ to } x \in [0,\infty). \tag{4.74}$$

This transformation is shown in Fig. 4.8. Under this substitution, the integral becomes:

$$\int_0^\infty f(x)\,dx = \int_0^1 \frac{f\left(\dfrac{t}{1-t}\right)}{(1-t)^2}\,dt. \tag{4.75}$$

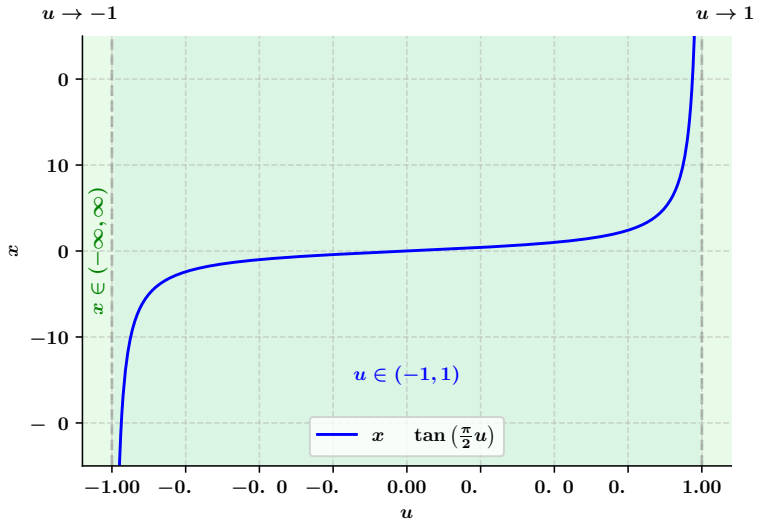

Figure 4.9.: Illustration of the transformation of an infinite interval.

Notice that when $t \to 1$, $x \to \infty$, and the integrand

$$\frac{f\left(\frac{t}{1-t}\right)}{(1-t)^2} \tag{4.76}$$

may become difficult or impossible to evaluate near $t = 1$ (for example, it may grow very large or become undefined). **Closed Newton-Cotes formulas** require evaluating the integrand at the endpoints $t = 0$ and $t = 1$. In this case, $t = 1$ corresponds to $x = \infty$, which is problematic. **Open Newton-Cotes formulas** use only interior points in $(0, 1)$. They *avoid evaluating the integrand at the potentially troublesome endpoints.*

4.4.4. Infinite integrals over the whole real line

When integrating over the entire real line:

$$\int_{-\infty}^{\infty} f(x)\, dx, \tag{4.77}$$

we can transform the infinite interval into a finite one. A common substitution is:

$$x = \tan\left(\frac{\pi}{2}u\right), \quad u \in (-1, 1), \tag{4.78}$$

which is shown in Fig. 4.9.This maps:

$$u \to -1 \quad \Rightarrow \quad x \to -\infty, \tag{4.79}$$

$$u \to 1 \quad \Rightarrow \quad x \to +\infty. \tag{4.80}$$

The differential transforms as:

$$dx = \frac{\pi}{2} \sec^2\left(\frac{\pi}{2}u\right)\, du. \tag{4.81}$$

Thus, the integral becomes:

$$\int_{-\infty}^{\infty} f(x)\, dx = \int_{-1}^{1} f\left(\tan\left[\frac{\pi}{2}u\right]\right) \cdot \frac{\pi}{2} \sec^2\left(\frac{\pi}{2}u\right)\, du. \tag{4.82}$$

Again, closed Newton-Cotes formulas would require evaluating the integrand at $u = -1$ and $u = 1$, which corresponds to $x = -\infty$ and $x = \infty$. **Open Newton-Cotes formulas** avoid these endpoints, so they prevent problems related to evaluating the function at infinity.

4.4.5. Advantages and Disadvantages

- Closed formulas are usually preferred when f is well-behaved and known at the boundaries. They are often slightly more accurate for the same degree because the interpolation covers the whole interval.
- Open formulas are used when:
  - Boundary points are problematic (unknown, infinite, ...)
  - or adaptive schemes want to avoid new evaluations at changing interval boundaries.
- Higher-degree Newton-Cotes formulas in general can suffer badly from Runge's phenomenon (oscillations near endpoints) unless used as composite rules.

4.5. Composite Numerical Integration

The Newton-Cotes formulas are typically not well suited for integration over large intervals. Using them would require high-degree polynomials, and determining the associated coefficients can become challenging. Moreover, Newton-Cotes formulas rely on interpolating polynomials with equally spaced nodes. For large intervals, this approach tends to be inaccurate due to the oscillatory behavior of high-degree polynomials.

In this section, we focus on a *piecewise* strategy for numerical integration. This method divides the interval into smaller subintervals and applies low-order Newton-Cotes formulas to each part. Such techniques are commonly used in practice. An illustration for an even number of intervals and piecewise application of Simpson's rule is shown in Fig. 4.10.

Example: Composite vs. Single Newton-Cotes Integration

Consider the function:

$$f(x) = \sin(x), \quad x \in [0, \pi/2]. \tag{4.83}$$

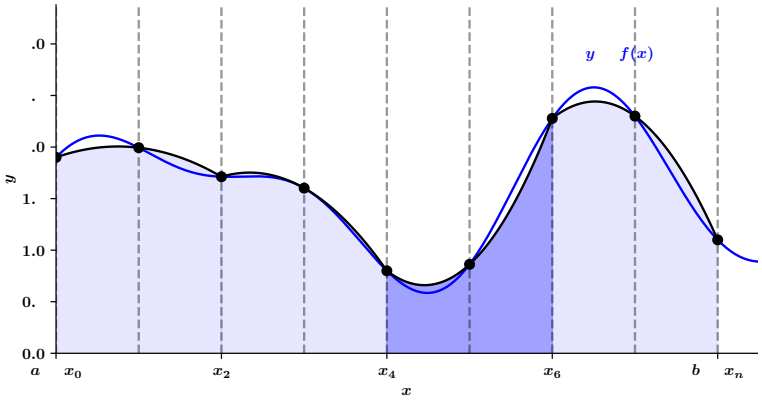

Figure 4.10.: Illustration of the composite integration applying Simpson's rule on an interval with even numbers of subintervals.

For simplicity, we select 5 equally spaced nodes:

$$x_0 = 0, \quad x_1 = \frac{\pi}{8}, \quad x_2 = \frac{\pi}{4}, \quad x_3 = \frac{3\pi}{8}, \quad x_4 = \frac{\pi}{2}. \tag{4.84}$$

The exact value of the integral is:

$$I = \int_0^{\pi/2} \sin(x)\, dx = 1. \tag{4.85}$$

1. Single Newton-Cotes Formula (n = 4, Closed — Boole's Rule):

The step size is:

$$h = \frac{\pi/2 - 0}{4} = \frac{\pi}{8}. \tag{4.86}$$

with $x_0 = 0, \ldots, x_4 = x_0 + 4h$ The fourth order NC approximation is:

$$I_{\text{Boole}} = \frac{2h}{45}\left[7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)\right]. \tag{4.87}$$

2. Composite Simpson's Rule (Two Segments):

The first segment is: $[x_0, x_2]$,

$$I_1 = \frac{h_1}{3}\left[f(x_0) + 4f(x_1) + f(x_2)\right], \quad h_1 = \frac{\pi}{8}. \tag{4.88}$$

The second one: $[x_2, x_4]$,

$$I_2 = \frac{h_2}{3}\left[f(x_2) + 4f(x_3) + f(x_4)\right], \quad h_2 = \frac{\pi}{8}. \tag{4.89}$$

Since the integral is a linear operation, the total is simply the sum over the two partial intervals,

$$I_{\text{Simpson}} = I_1 + I_2. \tag{4.90}$$

Results:

When evaluating the two integrals numerically, we find

$$I_{\text{Boole}} \approx 0.9998, \quad I_{\text{Simpson}} \approx 0.9999, \tag{4.91}$$

so both are very close to the exact value $I = 1$, with relative errors less than 2×10^{-4}.

4.5.1. Composite Simpson's Rule

To approximate the integral

$$I = \int_a^b f(x)\,dx \tag{4.92}$$

we simply chose an even integer n to subdivide the intervl $[a, b]$ in n subintervals and apply Simpson's rule on each consecutive pair of subintervals, see Fig. 4.10. We set $h = \frac{b-a}{n}$ and $x_j = a + jh$, for each $j = 0, 1, \ldots, n$ and find

$$\int_a^b f(x)\,dx = \sum_{j=1}^{n/2} \int_{x_{2j-2}}^{x_{2j}} f(x)\,dx \tag{4.93}$$

$$= \sum_{j=1}^{n/2} \left\{ \frac{h}{3} \left[f(x_{2j-2}) + 4f(x_{2j-1}) + f(x_{2j}) \right] - \frac{h^5}{90} f^{(4)}(\xi_j) \right\}. \tag{4.94}$$

Here again, the equation is exact for some ξ_j with $x_{2j-2} < \xi_j < x_{2j}$, under the condition that $f \in C^4[a, b]$. For each $j = 1, 2, \ldots, (n/2) - 1$ we have $f(x_{2j})$ appearing in the term corresponding to the interval $[x_{2j-2}, x_{2j}]$ and also in the term corresponding to the interval $[x_{2j}, x_{2j+2}]$, we can reduce this sum to

$$\int_a^b f(x)\,dx = \frac{h}{3} \left[f(x_0) + 2 \sum_{j=1}^{(n/2)-1} f(x_{2j}) + 4 \sum_{j=1}^{n/2} f(x_{2j-1}) + f(x_n) \right] - \frac{h^5}{90} \sum_{j=1}^{n/2} f^{(4)}(\xi_j). \tag{4.95}$$

Let $f \in C^4[a, b]$, n be even, $h = \frac{b-a}{n}$, and $x_j = a + jh$, for each $j = 0, 1, \ldots, n$. For the total integral we can find a $\mu \in (a, b)$ for which the **Composite Simpson's rule** for n subintervals can be written with its error term as

$$\int_a^b f(x)\,dx = \frac{h}{3} \left[f(a) + 2 \sum_{j=1}^{(n/2)-1} f(x_{2j}) + 4 \sum_{j=1}^{n/2} f(x_{2j-1}) + f(b) \right] - \frac{b-a}{180} h^4 f^{(4)}(\mu). \tag{4.96}$$

Algorithm The following algorithm illustrates the steps taken in the code:

INPUT: endpoints a, b; even positive integer n.
OUTPUT: approximation XI to I.

(step 1) Set $h = (b - a)/n$.

(step 2) Set
$XI0 = f(a) + f(b)$;
$XI1 = 0$; (Summation of $f(x_{2i-1})$.)
$XI2 = 0$; (Summation of $f(x_{2i})$.)

(step 3) For $i = 1, \ldots, n - 1$ do Steps 4 and 5:

(step 4) Set $X = a + ih$.

(step 5) If i is even
then
set $XI2 = XI2 + f(X)$;
else
set $XI1 = XI1 + f(X)$.

(step 6) Set

$$XI = \frac{h}{3}\left(XI0 + 2 \cdot XI2 + 4 \cdot XI1\right). \tag{4.97}$$

(step 7) OUTPUT (XI); STOP.

4.5.2. Lower order composite rules

For completeness we also show the composite Trapezoidal rule and the composite piecewise constant left-sided Riemann integration in Fig. 4.11. The corresponding total integral with the error for the **Composite Trapezoidal Rule** is

$$\int_a^b f(x)\,dx = \frac{h}{2}\left[f(a) + 2\sum_{j=1}^{n-1} f(x_j) + f(b)\right] - \frac{b-a}{12}h^2 f''(\mu) \tag{4.98}$$

for $\mu \in (a, b)$ over n subintervals.

4.5.3. Local vs. global error in composite integration

When applying numerical integration over an interval $[a, b]$, we typically approximate the integral locally on small subintervals via e.g., midpoint, trapezoidal, Simpson, and then combine these to obtain the global result. In terms of the error, we need to distinguish between a local error and the global counterpart:

- The **local truncation error** refers to the error made by applying the quadrature rule on a single subinterval of size h.

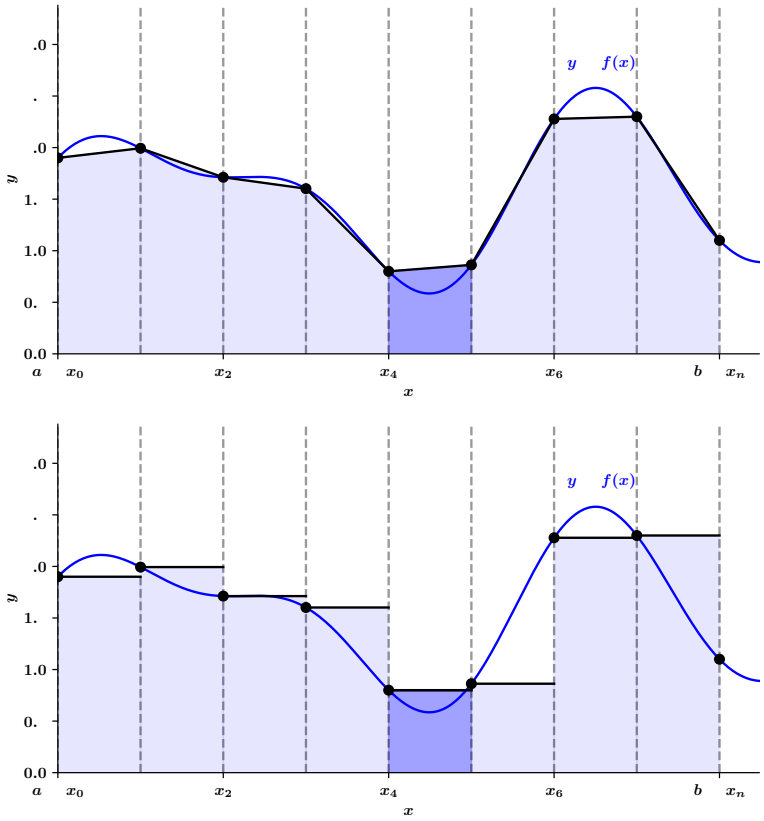

Figure 4.11.: Illustration of the composite trapezoidal rule and the composite Riemann intergral.

- The **global error** refers to the accumulated error over all such subintervals covering the entire domain $[a, b]$.

Suppose a quadrature rule applied on one subinterval of size h has local error

$$E_{\text{local}} = \mathcal{O}(h^p) \quad \text{(per subinterval)}. \tag{4.99}$$

If the interval $[a, b]$ is divided into $n = (b-a)/h$ subintervals, then the global (composite) error is approximately

$$E_{\text{global}} = n \cdot \mathcal{O}(h^p) = \mathcal{O}(h^{p-1}). \tag{4.100}$$

Thus, the global error is typically **one order lower** than the local error because the local errors accumulate over $\sim 1/h$ subintervals. This idea holds as long as the error per subinterval behaves regularly and no error cancellations occur globally.

Method	**Local Error**	**Global Error**
Trapezoidal Rule	$\mathcal{O}(h^3)$	$\mathcal{O}(h^2)$
Simpson's Rule	$\mathcal{O}(h^5)$	$\mathcal{O}(h^4)$

4.6. Richardson Extrapolation

Richardson extrapolation is a powerful technique in numerical analysis used to improve the accuracy of approximations, particularly in numerical integration and differentiation. It achieves this by combining approximations computed with different step sizes to eliminate leading-order error terms.

4.6.1. Mathematical Background

Suppose we aim to approximate a quantity A^* (such as an integral or derivative) using a numerical method that depends on the step size h. Let $A(h)$ denote this approximation. Often, the error in $A(h)$ can be expressed as a power series in h:

$$A(h) = A^* + C_1 h^{k_1} + C_2 h^{k_2} + C_3 h^{k_3} + \cdots, \tag{4.101}$$

where:

- A^* is the exact value we would like to compute,
- C_i are real constants,
- $k_1 < k_2 < k_3 < \cdots$ are known exponents indicating the order of the error terms.

Our goal is to eliminate the leading error term $C_1 h^{k_1}$ to obtain a more accurate approximation of A^*.

4.6.2. Richardson Extrapolation Formula

To eliminate the leading error term, we compute two approximations:

- $A(h)$: using step size h,
- $A(h/t)$: using a smaller step size h/t, where $t > 1$ (commonly $t = 2$).

Assuming the error behaves as above, we can combine these two approximations to cancel out the h^{k_1} term:

$$A_{\text{extrapolated}} = \frac{t^{k_1} A(h/t) - A(h)}{t^{k_1} - 1}. \tag{4.102}$$

Derivation

From the error expansions:

$$\begin{aligned} A(h) &= A^* + C_1 h^{k_1} + C_2 h^{k_2} + \cdots, \\ A(h/t) &= A^* + C_1 (h/t)^{k_1} + C_2 (h/t)^{k_2} + \cdots. \end{aligned} \tag{4.103}$$

Multiply the second equation by t^{k_1} and subtract the first:

$$\begin{aligned} t^{k_1} A(h/t) - A(h) &= t^{k_1} A^* + C_1 h^{k_1} + \cdots - A^* - C_1 h^{k_1} - \cdots \\ &= (t^{k_1} - 1) A^* + \text{higher-order terms}. \end{aligned} \tag{4.104}$$

Solving for A^*:

$$A^* = \frac{t^{k_1} A(h/t) - A(h)}{t^{k_1} - 1} + \text{higher-order terms} \tag{4.105}$$

$$= A(h/t) + \frac{A(h/t) - A(h)}{t^{k_1} - 1} + \text{higher-order terms} \tag{4.106}$$

Thus, $A_{\text{extrapolated}}$ provides a more accurate approximation of A^*, with the leading error term $C_1 h^{k_1}$ eliminated.

4.6.3. Recursive Richardson Extrapolation

The process can be applied recursively to eliminate successive error terms. Define:

$$\begin{aligned} A_1(h) &= A(h), \\ A_2(h) &= \frac{t^{k_1} A_1(h/t) - A_1(h)}{t^{k_1} - 1}, \\ A_3(h) &= \frac{t^{k_2} A_2(h/t) - A_2(h)}{t^{k_2} - 1}, \\ &\vdots \end{aligned}$$

Each application increases the order of accuracy by eliminating the next leading error term.

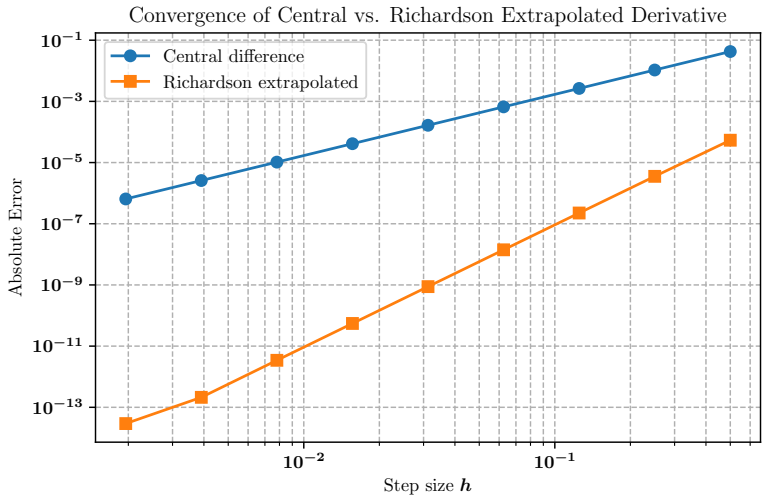

Figure 4.12.: Comparison of second order central differencing ($\propto h^2$) and Richardson extrapolation ($\propto h^4$).

Example: Richardson Extrapolation for Derivatives

Consider the function

$$f(x) = \sin(x) \cdot e^{-x}, \quad \text{with exact derivative} \quad f'(x) = \cos(x)e^{-x} - \sin(x)e^{-x}. \tag{4.107}$$

We approximate the derivative at a point x_0 using:

Central Difference (2nd order):

$$D(h) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} = f'(x_0) + C_1 h^2 + C_2 h^4 + \cdots \tag{4.108}$$

Richardson Extrapolation (4th order): Using $D(h)$ and $D(h/2)$ with $t = 2$, $k_1 = 2$, we define

$$D_{\text{rich}}(h) = \frac{4D(h/2) - D(h)}{3} = f'(x_0) + \mathcal{O}(h^4) \tag{4.109}$$

Observation: This procedure cancels the leading-order error term ($\propto h^2$), significantly improving the accuracy, see Fig. 4.12. For sufficiently smooth functions, Richardson extrapolation accelerates convergence and allows better accuracy at larger h, reducing numerical round-off issues.

4.7. Romberg Integration

Romberg integration is a systematic procedure that applies Richardson extrapolation to the composite trapezoidal rule to obtain increasingly accurate approximations to a

definite integral. To simplify the process and the notation we restrict the derivation to intervals that we cut in half for every further subdivision ($h \to h/2$) and we only use the Trapezoidal Rule.

Trapezoidal Rule and Error Structure

Let

$$I = \int_a^b f(x)\,dx \tag{4.110}$$

be approximated by the composite trapezoidal rule with step size h:

$$T(h) = \frac{h}{2}\left[f(a) + 2\sum_{i=1}^{n-1} f(a+ih) + f(b)\right] \tag{4.111}$$

The error of this approximation can be expressed as an even-power series:

$$T(h) = I + C_1 h^2 + C_2 h^4 + C_3 h^6 + \cdots \tag{4.112}$$

This structure makes the trapezoidal rule especially well-suited for Richardson extrapolation.

Richardson Extrapolation Applied

We denote the trapezoidal approximations for $n = 1, 2, 4, 8, \ldots$ by $R_{1,1}, R_{2,1}, R_{3,1}, R_{4,1}, \ldots$, each with an error of $\mathcal{O}(h^2)$. We then apply Richardson extrapolation to achieve approximations with errors $\mathcal{O}(h^4)$, denoted by $R_{2,2}, R_{3,2}, R_{4,2}, \ldots$. Using two approximations $T(h)$ and $T(h/2)$, the first Romberg integrals read

$$R_{1,1} = T(h) + \mathcal{O}(h^2) \tag{4.113}$$

$$R_{2,1} = T(h/2) + \mathcal{O}(h^2) \tag{4.114}$$

$$R_{2,2} = \frac{4T(h/2) - T(h)}{3} + \mathcal{O}(h^4) \tag{4.115}$$

$$\vdots \tag{4.116}$$

More generally, we define the Romberg extrapolated values as

$$R_{k,m} = R_{k,m-1} + \frac{R_{k,m-1} - R_{k-1,m-1}}{4^m - 1} \tag{4.117}$$

$$= \frac{4^m R_{k,m-1} - R_{k-1,m-1}}{4^m - 1} \tag{4.118}$$

This process recursively eliminates the leading error term of order h^{2m}.

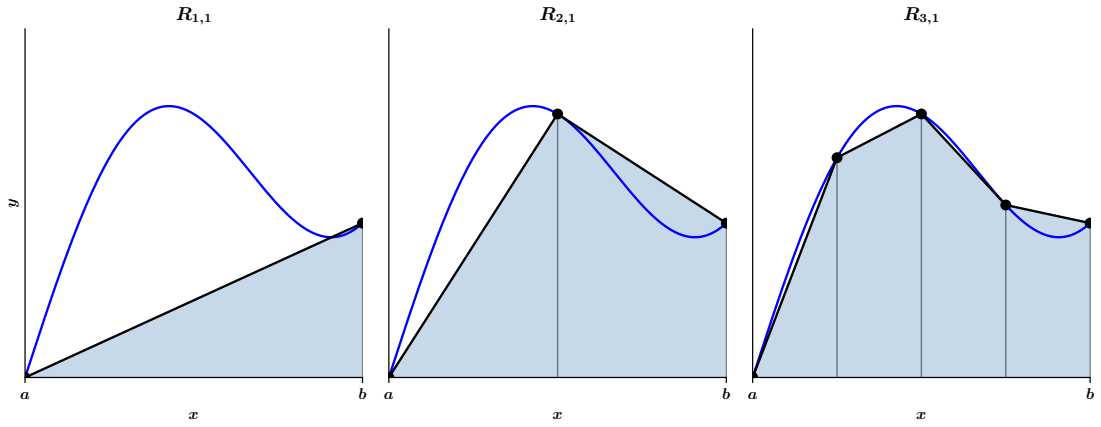

Figure 4.13.: Illustration of the Romberg $R_{i,1}$ terms. In each new iteration, only the missing intermediate function evaluations need to be computed.

Romberg Table Construction

By subsequently applying integrals and the Richardson extrapolations, we arrive at the Romberg table

$$\begin{array}{c|cccc} h & R_{1,1} & & & \\ h/2 & R_{2,1} & R_{2,2} & & \\ h/4 & R_{3,1} & R_{3,2} & R_{3,3} & \\ h/8 & R_{4,1} & R_{4,2} & R_{4,3} & R_{4,4} \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{array} \tag{4.119}$$

where again

- $R_{k,1}$ is the composite trapezoidal rule with 2^k intervals.
- $R_{k,m}$ for $m > 1$ is obtained from Richardson extrapolation, see above.

The table is constructed row-by-row in the order $R_{1,1}$, $R_{2,1}$, $R_{2,2}$, $R_{3,1}$, $R_{3,2}$, $R_{3,3}$, ... The construction Eq. (4.117) shows that this permits all computations in a new row to be computed by simple averaging over previously computed values plus *one additional* application of the Composite Trapezoidal Rule, see Fig. 4.13.

In terms of the points to evaluate, each consecutive approximation includes all the function's evaluations from the previous approximation. That is, $R_{1,1}$ used values at a and b, $R_{2,1}$ used these evaluations and added an additional evaluation at the intermediate point $(a + b)/2$. Then $R_{3,1}$ used the evaluations of $R_{2,1}$ and added two additional intermediate ones at $(a + b)/4$ and $3(a + b)/4$.

Key Points:

- Every $R_{k,m}$ approximates the same integral.

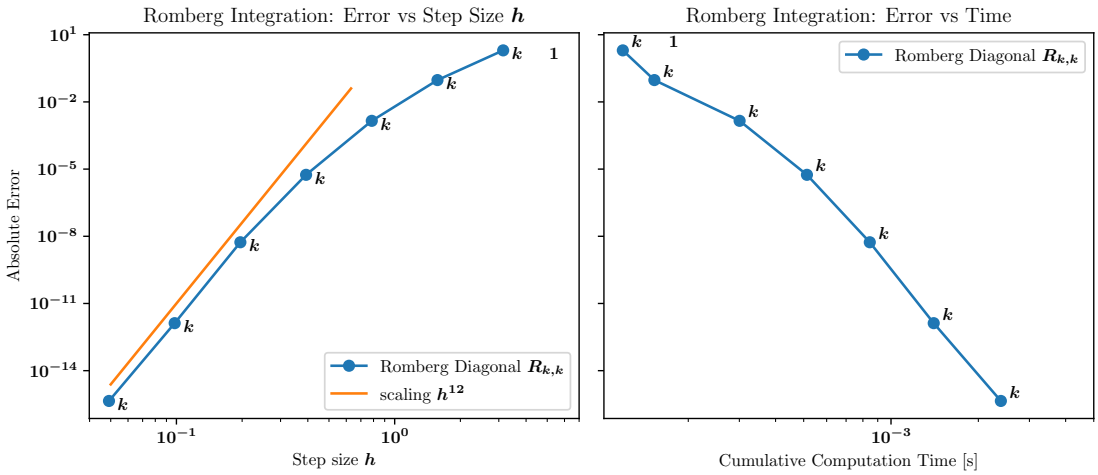

Figure 4.14.: Illustration of the error as a function of step size (left) as well as the error as a function of total computational time (right).

- Accuracy increases with both k and m, but the most accurate values are on the **diagonal** $R_{k,k}$, which eliminate all lower-order error terms.
- The typical convergence path is:

$$R_{1,1} \to R_{2,2} \to R_{3,3} \to R_{4,4} \to \cdots \tag{4.120}$$

- The stopping criterion is often based on:

$$|R_{k,k} - R_{k-1,k-1}| < \text{tolerance} \tag{4.121}$$

- Romberg integration is efficient because it reuses previously computed trapezoidal values.

Advantages and Limitations

- **Pros:** High-order convergence with minimal function evaluations for smooth integrands.
- **Cons:** Not well suited for functions with singularities, discontinuities, or oscillations.

4.7.1. Example

Here is an example of a Romberg integration table for the integral

$$\int_0^{\pi} \sin(x)\,\mathrm{d}x = 2 \tag{4.122}$$

showing both the Romberg estimates $R_{k,m}$ and their absolute errors.

Values :

error	$\mathcal{O}(h_k^2)$	$\mathcal{O}(h_k^4)$	$\mathcal{O}(h_k^6)$	$\mathcal{O}(h_k^8)$	$\mathcal{O}(h_k^{10})$
k/m	1	2	3	4	5
1	0.0000000000				
2	1.5707963268	2.0943951024			
3	1.8961188979	2.0045597550	1.9985707318		
4	1.9742316019	2.0002691699	1.9999831309	2.0000055500	
5	1.9935703438	2.0000165910	1.9999997525	2.0000000163	1.9999999946

Error :

error	$\mathcal{O}(h_k^2)$	$\mathcal{O}(h_k^4)$	$\mathcal{O}(h_k^6)$	$\mathcal{O}(h_k^8)$	$\mathcal{O}(h_k^{10})$
k/m	1	2	3	4	5
1	2.00e+00				
2	4.29e-01	9.44e-02			
3	1.04e-01	4.56e-03	1.43e-03		
4	2.58e-02	2.69e-04	1.69e-05	5.55e-06	
5	6.43e-03	1.66e-05	2.48e-07	1.63e-08	5.41e-09

The measured scaling of the error with step size as well as the error as a function of computational time are shown in Fig. 4.14

Romberg Integration Algorithm

To approximate the integral $I = \int_a^b f(x)\,dx$, select an integer $n > 0$.

- **INPUT:** endpoints a, b; integer n.
- **OUTPUT:** an array R. (Compute R by rows; only the last two rows are saved in storage.)

(step 1) Set $h = b - a$;

$$R_{1,1} = \frac{h}{2}\left(f(a) + f(b)\right). \tag{4.123}$$

(step 2) **OUTPUT** $R_{1,1}$.

(step 3) For $i = 2, \ldots, n$, do Steps 4–8.

(step 4) Set

$$R_{2,1} = \frac{1}{2}\left[R_{1,1} + h\sum_{k=1}^{2^{i-2}} f\left(a + \left(k - \frac{1}{2}\right)h\right)\right]. \tag{4.124}$$

(Approximation from Trapezoidal method.)

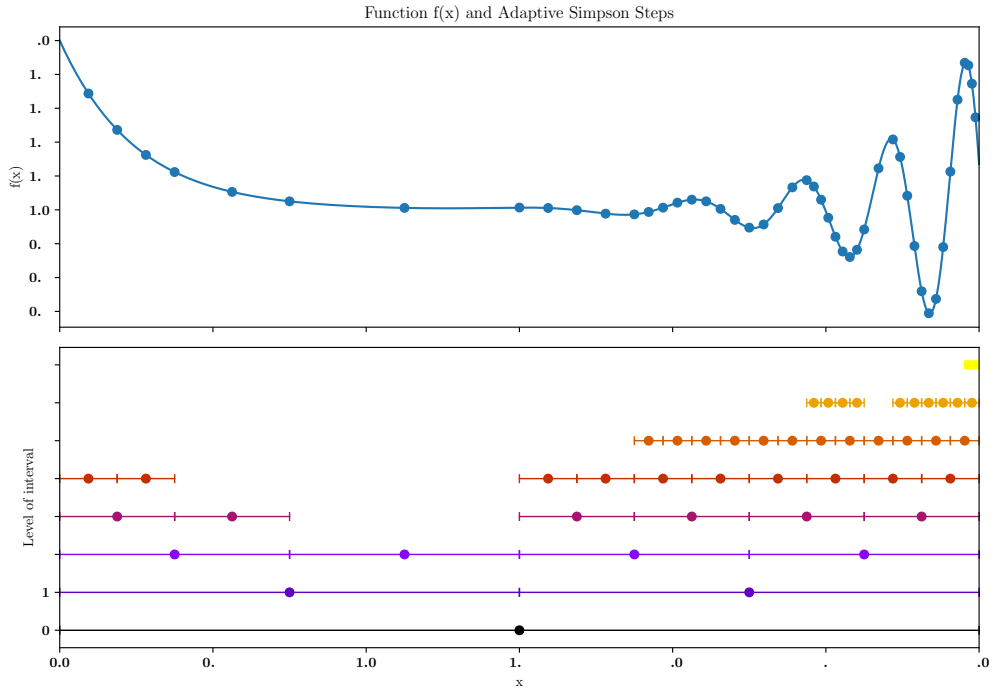

Figure 4.15.: Adaptive Simpson method with adaptively chosen step size based on the error between two approximations with half step size. The top panel shows the function, the bottom one the midpoints of the interval as well as the level (depth) of the recursive function call.

(step 5) For $j = 2, \ldots, i$, set

$$R_{2,j} = R_{2,j-1} + \frac{R_{2,j-1} - R_{1,j-1}}{4^{j-1} - 1}. \tag{4.125}$$

(Extrapolation.)

(step 6) **OUTPUT** $R_{2,j}$ for $j = 1, 2, \ldots, i$.

(step 7) Set $h = h/2$.

(step 8) For $j = 1, 2, \ldots, i$, set $R_{1,j} = R_{2,j}$.
(Update row 1 of R – only for efficient storage.)

(step 9) **STOP.**

4.8. Adaptive Quadrature Methods

We discussed composite integration methods, which are powerful as long as a simple spacing is a good approximation. However, they become inappropriate when the integrand has regions of strong changes as well as regions with little changes. Then the

narrow spacing required for the more dynamic part lead to unnecessary computational cost over the boring range of the function. So, how can we determine what step size and technique should be applied on various portions of the interval of integration. Also, how accurate can we expect the final approximation to be?

We can derive accuracy estimates given that the function we would like to integrate is *well behaved*. In case the error is evenly distributed in the interval $[a, b]$ over which we integrate, then a smaller step size is needed in regions, in which the function has large variations and vice versa. We note that large variations does not really need to simply mean *large changes* in f. A steep linear function can still be very nicely approximated by the trapezoidal approximation as long as it behaves linearly. A good method for this kind of problem should be able to detect how much the function varies and adjust the step size accordingly.

In this section, we explore an adaptive quadrature approach that not only helps reduce the approximation error but also provides an estimate of that error – without requiring higher derivatives of the function. While the technique is built around the Composite Simpson's rule, it can be easily adapted to work with other composite integration methods.

We need to approximate $\int_a^b f(x)\,dx$ with a maximum specified tolerance $\varepsilon > 0$. We first apply Simpson's rule with step size $h = \frac{b-a}{2}$. This gives

$$\int_a^b f(x)\,dx = S(a,b) - \frac{h^5}{90} f^{(4)}(\xi), \quad \text{for some } \xi \in (a,b). \tag{4.126}$$

with the approximation for the full interval $[a, b]$

$$S(a,b) = \frac{h}{3}\left[f(a) + 4f(a+h) + f(b)\right]. \tag{4.127}$$

Next, we need to determine the accuracy of the approximation. We know that the error terms depends on $f^{(4)}$, but we would like to avoid the evaluation of the fourth derivative. Instead, we apply the Composite Simpson's rule with $n = 4$ and step size $(b-a)/4 = h/2$, which yields

$$\begin{aligned}\int_a^b f(x)\,dx =& \frac{h}{6}\left[f(a) + 4f\left(a + \frac{h}{2}\right) + 2f(a+h) + 4f\left(a + \frac{3h}{2}\right) + f(b)\right] \\ &- \left(\frac{h}{2}\right)^4 \frac{(b-a)}{180} f^{(4)}(\tilde{\xi}),\end{aligned} \tag{4.128}$$

for some $\tilde{\xi} \in (a, b)$. We simplify the notation by defining $m = (a + b)/2$ and write

$$S\,(a,m) = \frac{h}{6}\left[f(a) + 4f\left(a + \frac{h}{2}\right) + f(a+h)\right] \tag{4.129}$$

and

$$S\,(m,b) = \frac{h}{6}\left[f(a+h) + 4f\left(a + \frac{3h}{2}\right) + f(b)\right]. \tag{4.130}$$

Then Eq. (4.128) can be rewritten as

$$\int_a^b f(x)\,dx = S\,(a,m) + S\,(m,b) - \frac{1}{16}\left(\frac{h^5}{90}\right) f^{(4)}(\xi). \tag{4.131}$$

To estimate the error we assume that $\xi \approx \tilde{\xi}$ or, more precisely, that $f^{(4)}(\xi) \approx f^{(4)}(\tilde{\xi})$. The success of this technique depends on the accuracy of this assumption. However, for a well behaved function the fourth deriative is likely to not behave in a complex way with poles, divergences or steps. In case this assumption holds, we can equate Eq. (4.126) and (4.131),

$$S\,(a,m) + S\,(m,b) - \frac{1}{16}\left(\frac{h^5}{90}\right) f^{(4)}(\xi) \approx S(a,b) - \frac{h^5}{90} f^{(4)}(\xi), \tag{4.132}$$

and we can estimate the error term

$$\frac{h^5}{90} f^{(4)}(\xi) \approx \frac{16}{15}\left[S(a,b) - S\,(a,m) - S\,(m,b)\right]. \tag{4.133}$$

Using this estimate in Eq. (4.131) produces an estimate for the error

$$\left|\int_a^b f(x)\,dx - S\,(a,m) - S\,(m,b)\right| \tag{4.134}$$

$$\approx \frac{1}{16}\left(\frac{h^5}{90}\right) f^{(4)}(\xi) \tag{4.135}$$

$$\approx \frac{1}{15}\left|S(a,b) - S\,(a,m) - S\,(m,b)\right|. \tag{4.136}$$

This means: $S(a,m) + S(m,b)$ approximates $\int_a^b f(x)\,dx$ approximately 15 times better than the value computed with one Simpson step on h, $S(a,b)$. Let ε be the given accuracy that we like to achieve. If

$$|S(a,b) - S\,(a,m) - S\,(m,b)| < 15\varepsilon, \tag{4.137}$$

then we expect

$$\left|\int_a^b f(x)\,dx - S\,(a,m) - S\,(m,b)\right| < \varepsilon. \tag{4.138}$$

Consequently,

$$S\,(a,m) + S\,(m,b) \tag{4.139}$$

is a sufficiently accurate approximation to $\int_a^b f(x)\,dx$.

4.8.1. Iterative algorithm

Based on the error estimate between a Simpson step over a step h and a composite Simpson step with two Simpson steps and half the interval step size each ($h/2$), we can construct an algorithm that locally ensures accuracy of a given ε. Set

(1) compute $S(a,b)$ and $S(a,m)+S(m,b)$

(2) check if $|S(a,b)-S(a,m)-S(m,b)|<15\varepsilon$

- if yes, accept $S(a,m)+S(m,b)$ as local solution
- if not,
  cut each interval ($[a,m]$ and $[m,b]$) in half
  cut error in half ($\varepsilon_2=\varepsilon/2$)
  repeat step (1) for both intervals with error check ε_2

This algorithm calls for a recursive implementation, which is shown in the following python snippet. The resulting adaptively chosen step sizes, i.e. the depth of the recursion is shown in Fig. 4.15

```
# Recursive adaptive Simpson integration
def adaptive_simpson(f, a, b, eps):
    m = (a + b) / 2
    S_ab = simpson(f, a, b)
    S_am = simpson(f, a, m)
    S_mb = simpson(f, m, b)
    delta = np.abs(S_ab - (S_am + S_mb))

    # Base case: Accept if error estimate is within tolerance
    if delta < 15 * eps:
        return S_am + S_mb
    else:
        # Recurse into [a, m] and [m, b] with half error tolerance
        left_result  = adaptive_simpson(f, a, m, eps / 2)
        right_result = adaptive_simpson(f, m, b, eps / 2)
        return left_result + right_result
```

4.9. Monte Carlo Integration

Monte Carlo integration estimates definite integrals using *random sampling*. It is particularly well-suited for high-dimensional problems, where classical quadrature becomes inefficient due to the large increase of function evaluations with increasing dimensionality.

4.9.1. Monte Carlo integration

Let $f : D \subset \mathbb{R}^d \to \mathbb{R}$ be an integrable function over a domain D. The goal is to compute:

$$I = \int_D f(x)\,dx \tag{4.140}$$

4.9.2. Uniform sampling in a bounded domain

Assume that the volume $V_D = \int_D dx$ is known, and that we can draw N independent random samples x_i uniformly distributed over D, i.e., $x_i \sim \mathcal{U}(D)$. Then the integral can be approximated by:

$$I \approx \sum_{i=1}^{N} f(x_i)\Delta V = \frac{V_D}{N} \sum_{i=1}^{N} f(x_i) \tag{4.141}$$

where:

- x_i: independent and identically distributed (i.i.d.) random samples drawn from the uniform distribution over D
- V_D: volume of the integration domain D
- N: total number of samples
- $\Delta V \approx V_D/N$: fraction of the domain that statistically belongs to $f(x_i)$

4.9.3. Derivation of the Error Estimate

Let $x_i \sim \mathcal{U}(D)$ be independent, uniformly distributed samples from the domain D, and define the random variable

$$X_i = f(x_i) \tag{4.142}$$

The Monte Carlo estimator is then:

$$\hat{I}_N = \frac{V_D}{N} \sum_{i=1}^{N} f(x_i) = V_D \cdot \bar{X}_N \tag{4.143}$$

where $\bar{X}_N$ is the sample mean of the X_i:

$$\bar{X}_N = \frac{1}{N} \sum_{i=1}^{N} X_i \tag{4.144}$$

Very generally, the variance scaling relation is quadratic,

$$\mathrm{Var}(aX) = a^2 \mathrm{Var}(X), \tag{4.145}$$

we obtain, so Since the X_i are independent and identically distributed (i.i.d.), the variance of the sample mean is:

$$\begin{aligned}
\mathrm{Var}(\bar{X}_N) &= \mathrm{Var}\left(\frac{1}{N}\sum_{i=1}^{N} X_i\right) && (4.146)\\
&= \frac{1}{N^2}\,\mathrm{Var}\left(\sum_{i=1}^{N} X_i\right) && (4.147)\\
&= \frac{1}{N^2}\sum_{i=1}^{N}\mathrm{Var}(X_i) && (4.148)\\
&= \frac{1}{N^2}\sum_{i=1}^{N}\mathrm{Var}(X_i) && (4.149)\\
&= \frac{1}{N^2}\,N\sigma_f^2 && (4.150)\\
&= \frac{\sigma_f^2}{N}. && (4.151)
\end{aligned}$$

where $\mathrm{Var}(X_i)$ denotes the variance of any of the i.i.d. random variables $X_i = f(x_i)$. This is often written as $\mathrm{Var}(X_1)$ in shorthand, since all X_i have the same distribution. The variance σ_f^2 of the function $f(x)$ over the domain D is:

$$\sigma_f^2 = \mathrm{Var}(f(x)) = \frac{1}{V_D}\int_D \left(f(x) - \langle f\rangle\right)^2 dx \tag{4.152}$$

with:

$$\langle f\rangle = \frac{1}{V_D}\int_D f(x)\,dx \tag{4.153}$$

Hence, the variance of the Monte Carlo estimate is:

$$\mathrm{Var}(\hat{I}_N) = V_D^2 \cdot \mathrm{Var}(\bar{X}_N) = \frac{V_D^2}{N}\,\sigma_f^2 \tag{4.154}$$

Taking the square root gives the standard deviation (error estimate):

$$\sigma_{\mathrm{MC}} = \sqrt{\mathrm{Var}(\hat{I}_N)} = \frac{V_D\sigma_f}{\sqrt{N}} \tag{4.155}$$

The important feature is that the error decreases as $\sim 1/\sqrt{N}$, *independent of the dimension d*. We note that σ_f is a property of the function f itself, not the approximated integral, which we do not need to compute explicitly.

4.9.4. Importance Sampling

More generally, for a probability distribution $p(x)$ supported on D (with $p(x) > 0$ where $f(x) \neq 0$), the integral can be rewritten as an expectation value:

$$I = \int_D f(x)\,dx = \int_D \frac{f(x)}{p(x)} p(x)\,dx = \mathbb{E}_p\left[\frac{f(x)}{p(x)}\right] \tag{4.156}$$

and approximated by:

$$I \approx \frac{1}{N}\sum_{i=1}^{N} \frac{f(x_i)}{p(x_i)} \quad \text{with} \quad x_i \sim p(x) \tag{4.157}$$

This formulation allows for *importance sampling*, where $p(x)$ is chosen to reduce the variance of the estimator by concentrating samples in regions where $f(x)$ is large.

4.9.5. Example: Estimating π

Monte Carlo Area Estimation in 2D

We estimate π by computing the area of a quarter unit circle inscribed in the unit square $[0,1]^2$. The area of this quarter circle is $\pi/4$.

We define the indicator function

$$\chi(x,y) = \begin{cases} 1 & \text{if } x^2 + y^2 \leq 1 \\ 0 & \text{otherwise} \end{cases} \tag{4.158}$$

and compute π via

$$\frac{\pi}{4} = \int_0^1 \int_0^1 \chi(x,y)\,dy\,dx. \tag{4.159}$$

We can estimate this using Monte Carlo integration by sampling N points (x_i, y_i) uniformly in $[0,1]^2$ to find

$$\pi \approx \frac{4}{N}\sum_{i=1}^{N} \chi(x_i, y_i) \quad \text{where} \quad x_i, y_i \sim \mathcal{U}(0,1). \tag{4.160}$$

Classical 1D Integral for Comparison

We can also estimate π using the integral over the upper half of the unit circle:

$$\pi = 4\int_0^1 \sqrt{1 - x^2}\,dx \tag{4.161}$$

This 1D integral is well-suited to classical numerical integration methods.

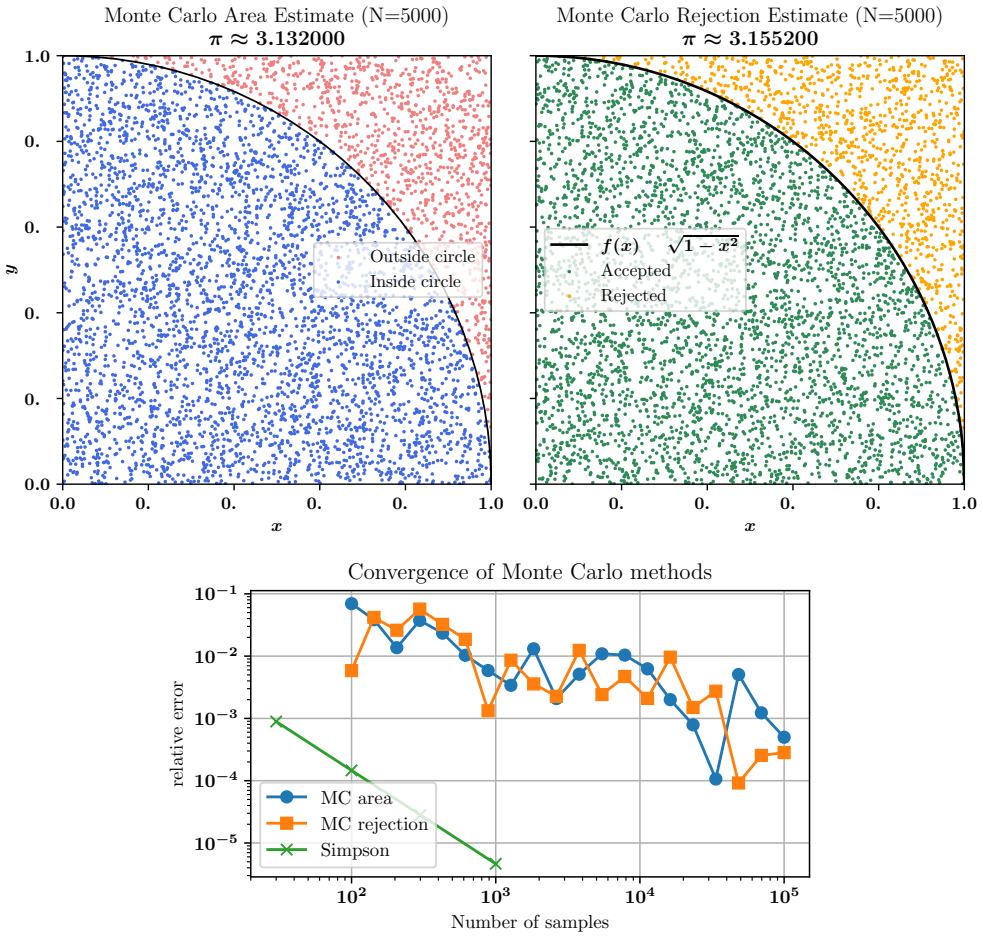

Figure 4.16.: Illustration of the computation of π using MC integration (top). The error scales with $\sqrt{N}$ and is for simple problems much worse than direct 1D computations (bottom)

Illustration and error scaling

We compare the MC approach and the relative error as a function of computational elements in Fig. 4.16. For this problem the scaling of the classical deterministic methods such as Simpson's rule is significantly better.

4.9.6. Accuracy of Monte Carlo Integration

General comparison: Monte Carlo integration has a dimension-independent convergence rate:

$$\text{Error}_{\text{MC}} \sim \mathcal{O}\left(\frac{1}{\sqrt{N}}\right) \tag{4.162}$$

This is in contrast to classical 1D methods such as the trapezoidal or Simpson's rule, which can achieve:

$$\text{Error}_{\text{trapezoidal}} \sim \mathcal{O}(N^{-2}), \quad \text{Error}_{\text{Simpson}} \sim \mathcal{O}(N^{-4}) \tag{4.163}$$

In low dimensions ($d \leq 4$), deterministic methods are generally *much more accurate* and faster for smooth integrands.

4.9.7. Generalization to d-Dimensions

We would like to illustrate that a fraction of a simple rectangular domain becomes more and more challenging when going to higher dimensions. As an example, we illustrate the fraction of the volume that a d-dimensional sphere occupies.

Let us consider the unit ball in d dimensions:

$$B_d = \left\{ x \in \mathbb{R}^d : \|x\|_2 \leq 1 \right\} \tag{4.164}$$

Its volume is given analytically by:

$$V_d = \frac{\pi^{d/2}}{\Gamma\left(\frac{d}{2}+1\right)} \quad \text{where } \Gamma \text{ is the generalized factorial} \tag{4.165}$$

Let us compare the volume of the unit ball B_d in $\mathbb{R}^d$ to the volume of the enclosing cube $[-1,1]^d$:

Dimension d	Volume V_d of Sphere	Volume 2^d of Cube	Fraction $V_d/2^d$
2	$\pi \approx 3.1416$	4	≈ 0.7854
3	$4\pi/3 \approx 4.1888$	8	≈ 0.5236
4	$\pi^2/2 \approx 4.9348$	16	≈ 0.3084

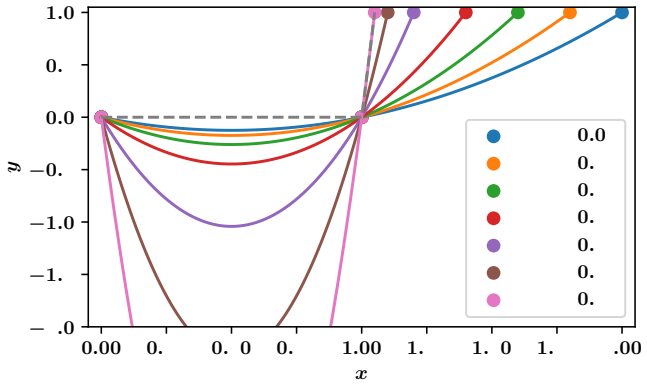

Figure 4.17.: Illustration of the problem with quadratic interpolation and integration in case of a step function. Higher resolution will make the step sharper, which causes a quadratic interpolant to create unwanted extremes that do not reflect the true function shape.

Whereas for classical methods we would need more and more regions with zero function values and with a complex boundary, The MC methods become superior. We estimate the volume (as the simplest example) by sampling uniformly from the enclosing d-dimensional cube $[-1,1]^d$ of volume $V_{\text{cube}} = 2^d$. We define the indicator function

$$\chi(x) = \begin{cases} 1 & \text{if } \|x\|_2 \leq 1 \\ 0 & \text{otherwise} \end{cases} \tag{4.166}$$

and the integral

$$V_d = \int_{[-1,1]^d} \chi(x)\, dx \approx V_{\text{cube}} \cdot \frac{1}{N} \sum_{i=1}^{N} \chi(x_i) \quad \text{with} \quad x_i \sim \mathcal{U}([-1,1]^d). \tag{4.167}$$

This method allows us to compute V_d numerically even for large d.

Good applications for Monte Carlo Monte Carlo becomes preferable or even necessary when:

- The number of dimensions d is large ($d \gtrsim 6$).
- The integration domain has a complex geometry.
- The integrand is noisy or discontinuous
  - classical error estimates do not work (derivative)
  - Trapezoidal Rule needs tiny h to match step
  - Simpson's Rule might cause unwanted extremes, see Fig. 4.17

- Easy to parallelize code (less book keeping)
- Special hardware / GPU acceleration is available.
- In high dimensions, Monte Carlo methods are often the *only* practical tool available. They are especially effective for computing expectations in Bayesian inference, particle physics, and statistical mechanics.

Example: Monte Carlo Integration Over a Complex Domain

Classical quadrature methods are well-suited to rectangular domains but struggle with irregular geometries. Monte Carlo methods, by contrast, can easily handle such domains using rejection sampling.

Setup: Consider the annulus:

$$D = \left\{ (x, y) \in \mathbb{R}^2 \;\middle|\; 0.5 \leq \sqrt{x^2 + y^2} \leq 1 \right\} \tag{4.168}$$

and the integral:

$$I = \iint_D \left(x^2 + y^2\right)\, dx\, dy \tag{4.169}$$

Monte Carlo method:

- Sample N points uniformly in $[-1, 1]^2$
- Accept only points where $0.5 \leq \sqrt{x^2 + y^2} \leq 1$
- Estimate:

$$I \approx 4 \cdot \frac{1}{N} \sum_{\text{accepted}} \left(x_i^2 + y_i^2\right) \quad \text{(scaled by square area)} \tag{4.170}$$

Why Monte Carlo wins: The irregular shape of the domain makes grid-based methods inefficient. Adapting the grid to the curved boundary is difficult, while Monte Carlo only requires a boolean condition to define the region.

4.9.8. Complex shape as simple function root

Suppose we need to solve a multi-dimensional problem with a complex boundary that is given by a physics limit, such as a given energy. As an example we use the 2D function

$$C(x, y) = x^2 + y^2 - 0.6 + 0.15\, \sin(5x) \cos(5y) \tag{4.171}$$

and are interested in the fraction of the domain for which $C(x, y)$ is negative, i.e.

$$D = \left\{ (x, y) \in \mathbb{R}^2 \;\middle|\; C(x, y) < 0 \right\} \tag{4.172}$$

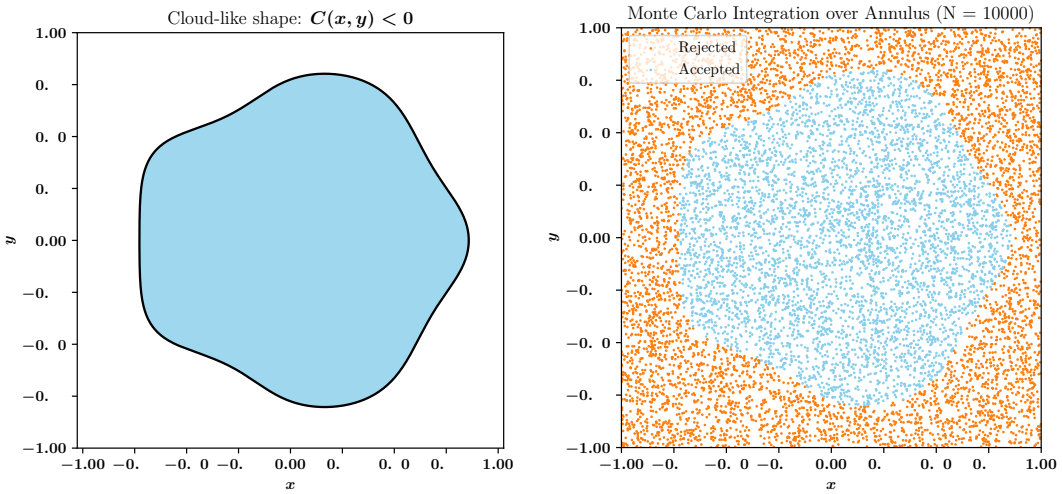

Figure 4.18.: Illustration of a complex shape of the integration domain, given by a function value $f(x, y) < 0$ (left). The corresponding MC integration points are shown on the right.

This gives a mathematically simple definition, which can be very complicated to describe analytically as boundary points for classical integration methods. In the MC case, we simply need an acceptance function (mask function), which tells us for a random point x_i, y_i if the condition $C(x_i, y_i) < 0$. If this condition is fulfilled, we count the point in the MC sum and weight it by the corresponding function value of the quantity we like to integrate $f(x, y)$. We illustrate the MC method of the cloud shape domain in Fig. 4.18.

Finally, here is a code snippet that illustrates this specific application:

```
# Define the integrand
def f(x, y):
    return x**2 + y**2

# Define domain function
def cloud_function(x, y):
    ff = x**2 + y**2 - 0.6 + 0.15 * np.sin(5 * x) * np.cos(5 * y)
    return ff < 0

# Monte Carlo integration
def monte_carlo(N, domain_function, amplitude_function):
    # Sample uniformly in the square [-1, 1]^2
    x = np.random.uniform(-1, 1, N)
    y = np.random.uniform(-1, 1, N)

    # Mask: inside the annulus
    mask = domain_function(x, y)
    x_in = x[mask]
    y_in = y[mask]
    num_inside = len(x_in)
```

```
    # Compute the function values for accepted points
    values = amplitude_function(x_in, y_in)
    mean_val = np.mean(values)

    # Estimate the integral: area of square is 4
    area_square = 4.0
    integral_est = area_square * mean_val * num_inside / N

    return integral_est, x_in, y_in, x[~mask], y[~mask]

# Run with N samples
N = 10_000
I_est, x_in, y_in, x_out, y_out = monte_carlo(N, cloud_function, f)
```

4.9.9. Parallelization of Monte Carlo Integration

Monte Carlo integration is particularly easy to parallelize. The reason is that the random samples are independent. Therefore, each processor can draw its own set of random points in the full integration domain and compute a partial sum.

Assume that we use P processors. Processor p draws N_p independent random points $x_{p,i} \sim \mathcal{U}(D)$, with

$$N = \sum_{p=1}^{P} N_p. \tag{4.173}$$

The Monte Carlo estimator can then be written as

$$\hat{I}_N = \frac{V_D}{N} \sum_{p=1}^{P} \sum_{i=1}^{N_p} f(x_{p,i}). \tag{4.174}$$

Equivalently, each processor computes a local sum

$$S_p = \sum_{i=1}^{N_p} f(x_{p,i}), \tag{4.175}$$

and the final result is obtained by a global reduction:

$$\hat{I}_N = \frac{V_D}{N} \sum_{p=1}^{P} S_p. \tag{4.176}$$

If all processors use the same number of samples, $N_p = N/P$, this becomes

$$\hat{I}_N = \frac{V_D}{P} \sum_{p=1}^{P} \left(\frac{1}{N_p} \sum_{i=1}^{N_p} f(x_{p,i}) \right). \tag{4.177}$$

Thus, the global estimate is simply the average of the local Monte Carlo estimates.

The variance of the parallel estimator is the same as for a serial Monte Carlo calculation with the same total number of samples:

$$\mathrm{Var}(\hat{I}_N) = \frac{V_D^2}{N}\sigma_f^2. \tag{4.178}$$

Parallelization therefore reduces the wall-clock time, but not the mathematical convergence rate. The error still decreases as

$$\sigma_{\mathrm{MC}} = \frac{V_D \sigma_f}{\sqrt{N}}. \tag{4.179}$$

Important implementation detail: Each processor must use an independent random number stream. In practice this means that the random number generator should be initialized with different seeds, or better, with a parallel random number generator that provides independent substreams.

Comparison with classical domain decomposition

For classical grid-based quadrature, the domain is usually decomposed geometrically. For example, in a two-dimensional trapezoidal integration the domain is split into rectangular subdomains, and each processor integrates only over its own part of the grid. This requires bookkeeping of grid boundaries and, depending on the method, communication of boundary values.

Monte Carlo integration is different. Each processor samples points throughout the full domain. There is no need to assign a specific spatial subdomain to a processor. The only required communication is the final reduction of the local sums S_p.

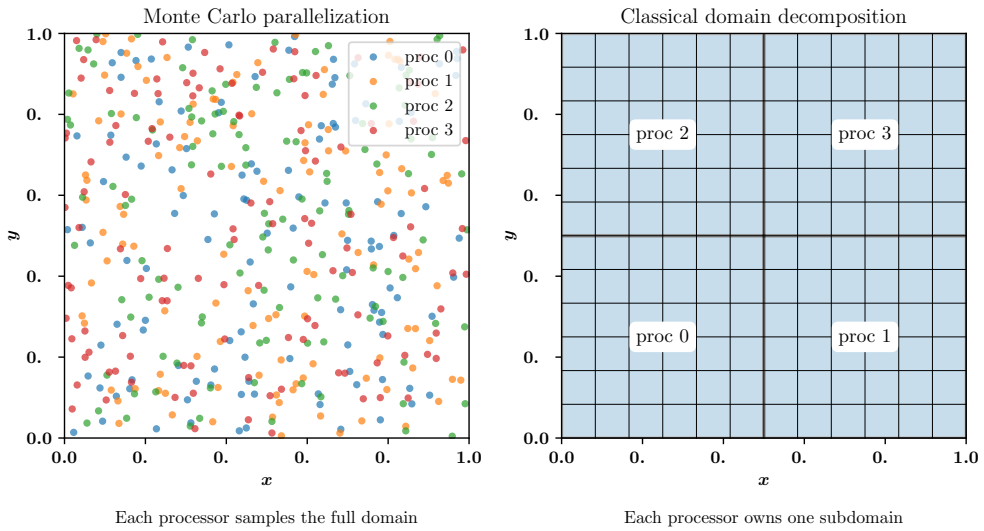

Figure 4.19.: Parallelization strategies for Monte Carlo integration and classical grid-based quadrature. In Monte Carlo integration, each processor samples random points in the full domain. In classical domain decomposition, the physical domain is split into subdomains, and each processor integrates its assigned region.